

Attention Mechanism

Natural Language Processing

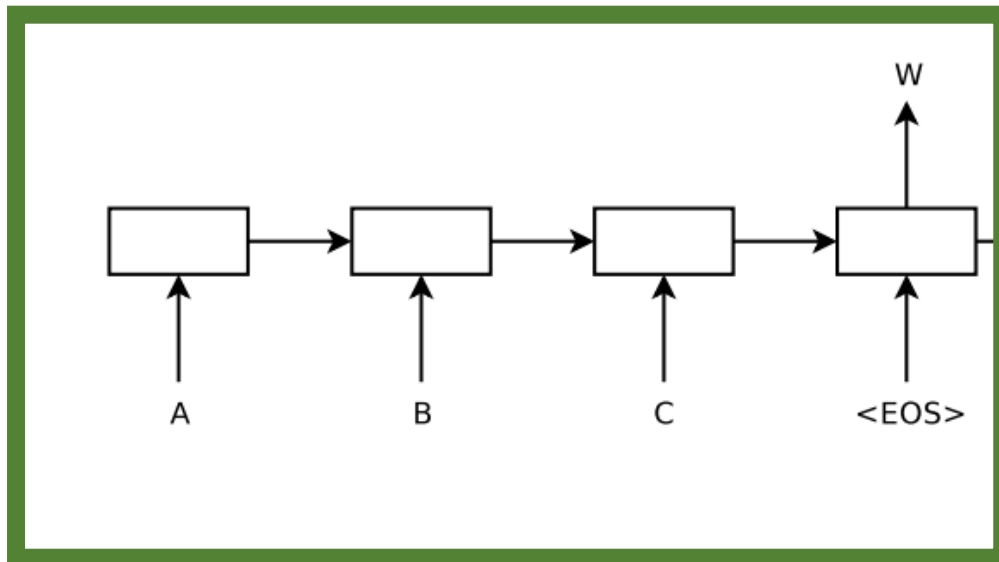
Ayesha Enayet

Background

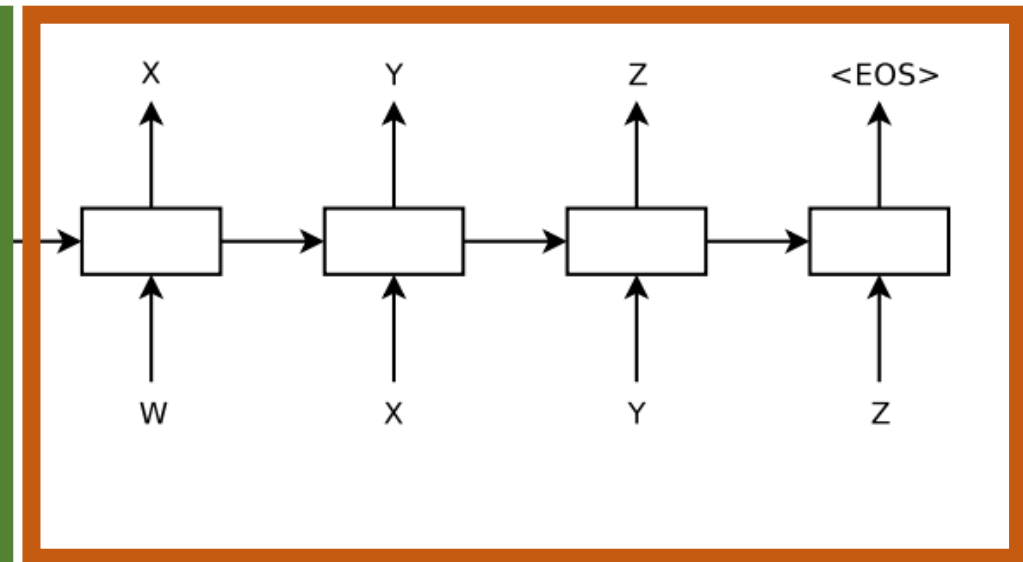
- The RNN can easily map sequences to sequences whenever the alignment between the inputs the outputs is known ahead of time.
- It doesn't work well for the problems whose input and the output sequences have different length.
- Sequences pose a challenge for RNNs because they require that the dimensionality of the inputs and outputs is known and fixed.

Encoder-Decoder Models

Encoder



Decoder



Sequence to Sequence Learning with Neural Networks: <https://arxiv.org/pdf/1409.3215>

RNN Encoder–Decoder ([Cho et al., 2014](#))

- One RNN encodes a sequence of symbols into a fixed length vector representation, and the other decodes the representation into another sequence of symbols (variable length).
- learn the conditional distribution over a variable-length sequence conditioned on yet another variable-length sequence, e.g. $p(y_1, \dots, y_T \mid x_1, \dots, x_T)$,

Drawbacks

- Drawback: Only final hidden state (context vector) passed on to the decoder.
- Solution: Attention.

Attention Mechanism

- Attention lets a model **dynamically focus on the most relevant parts of the input** when generating each output token, instead of squeezing the whole input into a single fixed vector. This reduces information loss from long sequences.

Attention (RNN based)

- First, the encoder passes *all* the hidden states to the decoder.
- Second, an attention decoder does an extra step before producing its output. In order to focus on the parts of the input that are relevant to this decoding time step, the decoder does the following:
 1. Look at the set of encoder hidden states it received – each encoder hidden state is most associated with a certain word in the input sentence
 2. Give each hidden state a score (let's ignore how the scoring is done for now)
 3. Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores

Attention Scores

- Can be computed:
 1. The attention score can be computed through cosine similarity, i.e., the dot product of two-word vectors, which assesses the strength of their relationship or the degree of similarity between the compared words.
 2. Feedforward Neural Network.

Context Vector: $C(i) = \sum_{j=1}^{T_x} \alpha(i, j) h(j)$

Score: $e_{i,j} = f(s_{i-1}, h_j)$

Attention Score (SoftMax of score): $\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^T \exp(e_{i,k})}$

$$\underbrace{e_{i,1}, e_{i,2}, \dots, e_{i,T_x}}_{\text{"all } e_{i,j} \text{ for this } i"}$$

Here's the full computation at decoder step i :

1. Scores (compatibility) for each encoder position

$$e_{i,j} = f(s_{i-1}, h_j) \quad \text{for } j = 1, \dots, T_x$$

(e.g., Bahdanau: $e_{i,j} = v_a^\top \tanh(W_s s_{i-1} + W_h h_j + b)$)

2. Normalize to attention weights (over j)

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^{T_x} \exp(e_{i,k})}$$

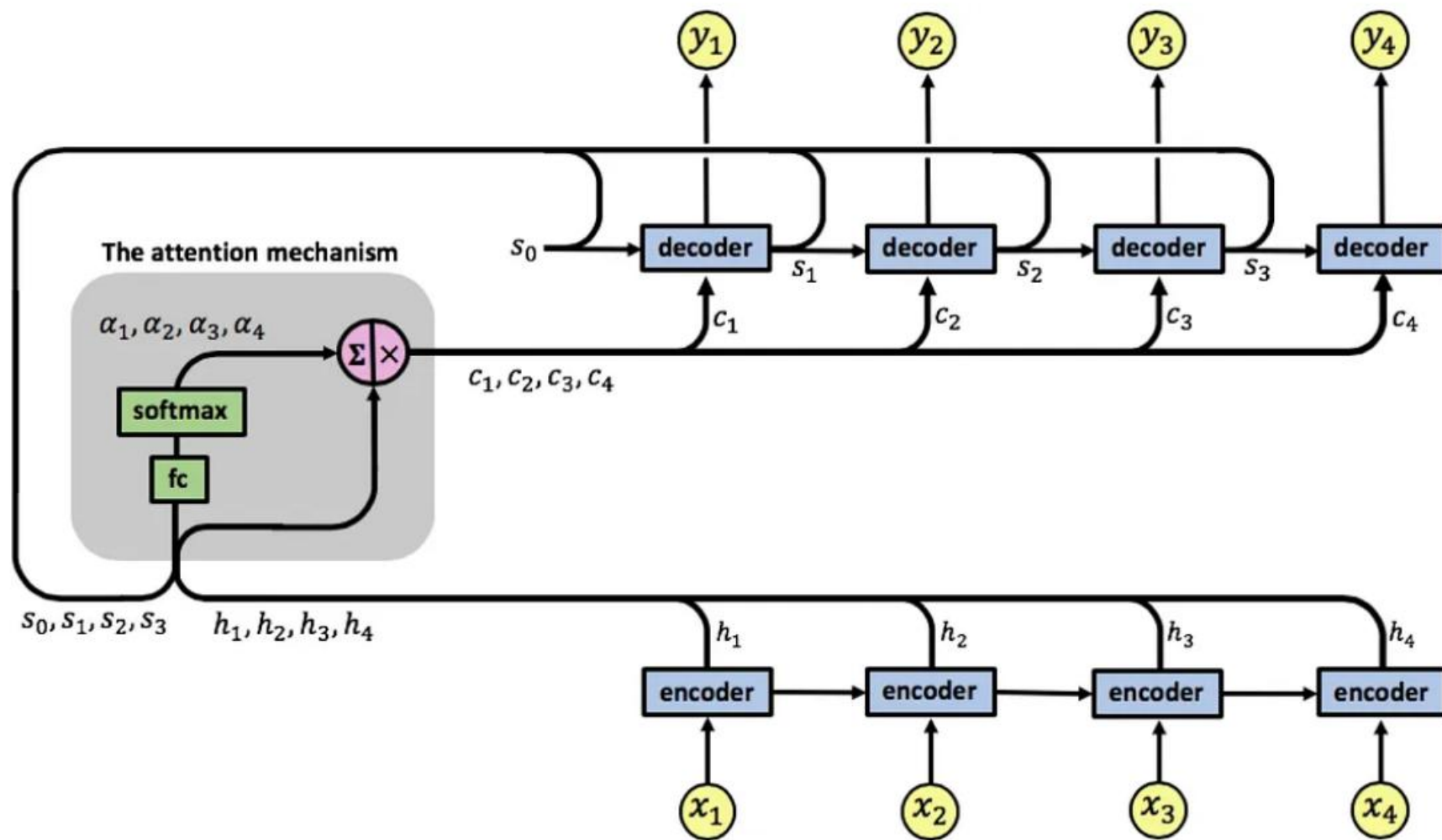
Example

- $S_{i-1}=100 \times 1$
- $H_j=100 \times 1$
- $W_s=I_1$ [e.g. identity matrix of dimension 128×100]
- $W_h=I_2$ [e.g. identity matrix of dimension 128×100]
- $W_s S_{i-1}=128 \times 1$
- $W_h H_j=128 \times 1$
- Weighted sum= 128×1
- $V_a=128 \times 1$

$$e_{i,j} = v_a^\top \tanh(W_s s_{i-1} + W_h h_j + b)$$

$$\underbrace{e_{i,1}, e_{i,2}, \dots, e_{i,T_x}}_{\text{"all } e_{i,j} \text{ for this } i"}$$

Symbol	Meaning	Shape
$\mathbf{s}_{i-1}$	previous decoder hidden state	(512,)
$\mathbf{h}_j$	encoder hidden state at position j	(100,)
$\mathbf{W}_s$	weight matrix for decoder state	(128, 512)
$\mathbf{W}_h$	weight matrix for encoder state	(128, 100)
$\mathbf{b}$	bias	(128,)
$\mathbf{v}_a$	attention projection vector	(128,)



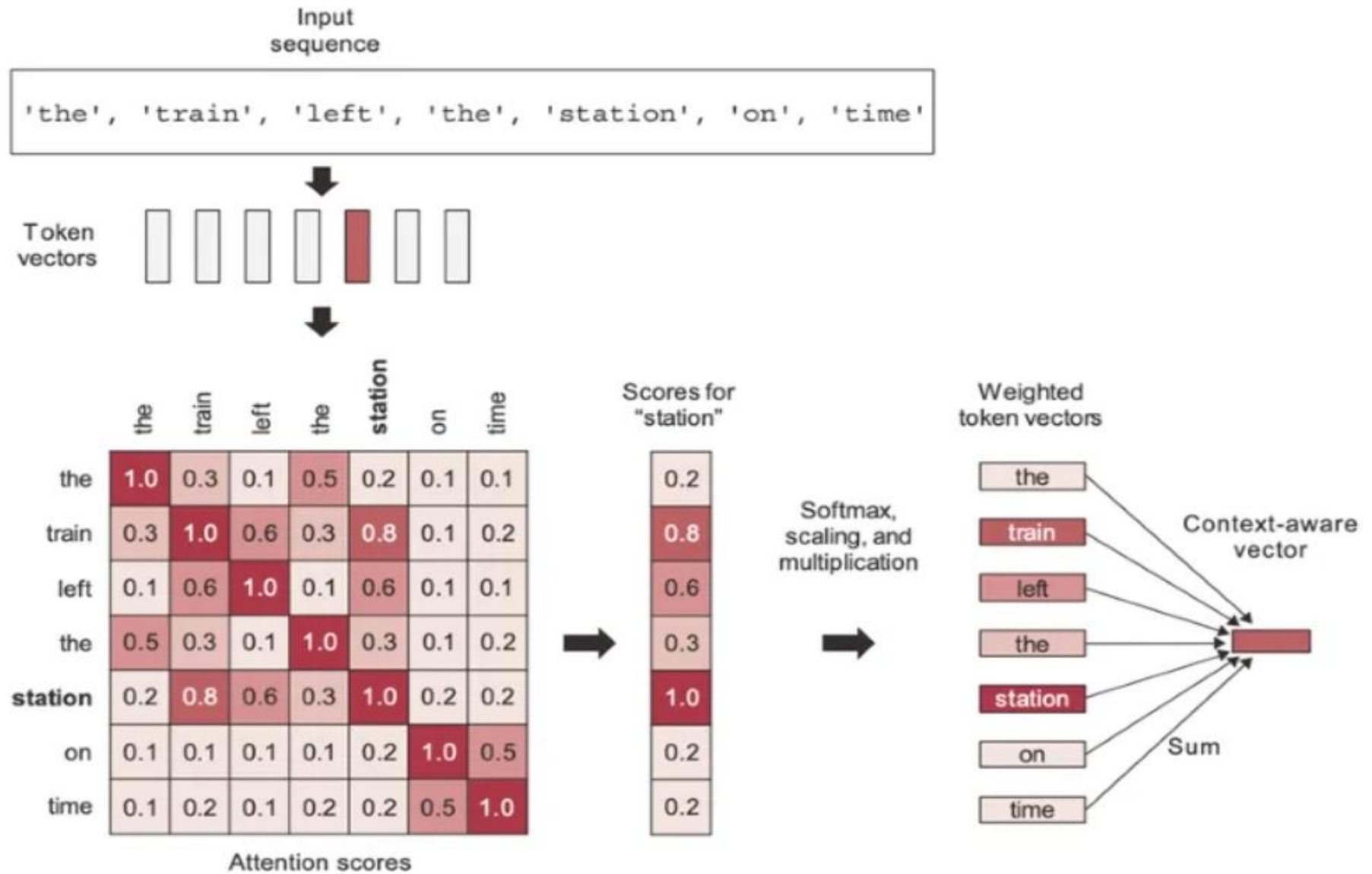
Types of Attention

- Self-Attention:
 - Focuses on relationships within the same sequence.
 - Encoder, Decoder
- Cross-Attention:
 - Enables the model to consider relationships between different sequences (e.g., input and output).
 - Decoder

Reference: [https://medium.com/@abhinavbhartigoml/unraveling-transformers-a-deep-dive-into-self-attention-and-3e37dc875bea#:~:text=Self%2DAttention%3A%20Focuses%20on%20relationships,e.g.%2C%20input%20and%20output\).](https://medium.com/@abhinavbhartigoml/unraveling-transformers-a-deep-dive-into-self-attention-and-3e37dc875bea#:~:text=Self%2DAttention%3A%20Focuses%20on%20relationships,e.g.%2C%20input%20and%20output).)

Self Attention

- [ht](#)
[d8](#)



Self Attention

- Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations.

Table 1: Comparison of Self attention and RNN based Attention

RNN Based Attention Mechanism	Self-Attention Mechanism
score: $e_{i,j} = f(s_{i-1}, h_j)$	score $= x_i^T x_j$ [Dot product of input vector with itself.]
Attention Score (SoftMax of score): $\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^T \exp(e_{i,k})}$	Attention score: $\alpha_{i,j} = \text{SoftMax}(\text{score}/\text{const.})$ $= \text{SoftMax} \left(\frac{x_i^T x_j}{\text{const}} \right)$ [Dividing by a constant to control the variance.]
Context Vector: $C(i) = \sum_{j=1}^{T_x} \alpha(i, j) h(j)$	Self-Attention Vector: $A(i) = \sum_{j=1}^{T_x} \alpha(i, j) x_j$ $= \text{SoftMax} \left(\frac{x_i^T x_j}{\text{const}} \right) \cdot x_j$ [in vectorized form]

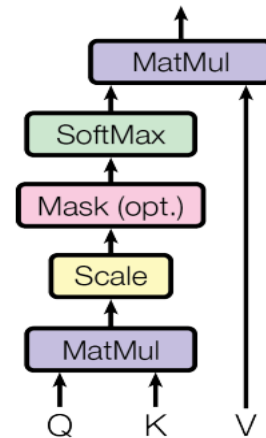
Drawback

- The drawback of previously proposed self-attention:
 - We computed the Self-Attention vector, as derived above, based on the inputs of the vectors themselves.
 - In other words, there are no learnable parameters.

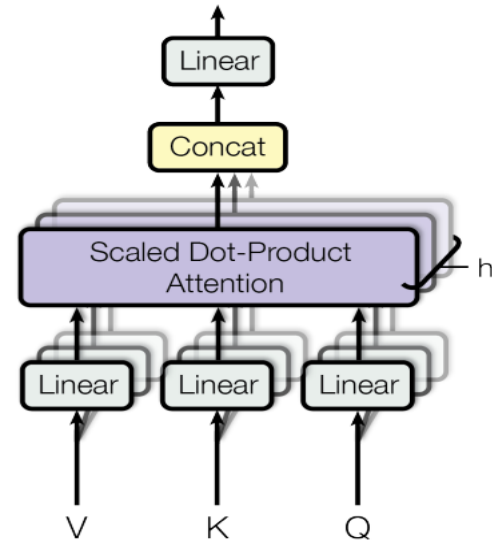
Solution

- “Attention is all you need”: **three trainable weight matrices are introduced and multiplied with input X_i separately, and three new terms Queries(Q), Keys(K), and Values(V) come into the picture.**

Scaled Dot-Product Attention



Multi-Head Attention



Solution

shape of $X \rightarrow (T \times d_{model})$

d_{model} = Embedding vector for each word (512 as per the paper)

Queries: $\mathbf{Q} = \mathbf{XW}^Q$; Where $\mathbf{W}^Q$ is weight matrix introduced to calculate Q from X.

Keys: $\mathbf{K} = \mathbf{XW}^K$; Where $\mathbf{W}^K$ is weight matrix introduced to calculate K from X.

Values: $\mathbf{V} = \mathbf{XW}^V$; Where $\mathbf{W}^V$ is weight matrix introduced to calculate V from X.

Shape tracking $\rightarrow$ (*shape of X*) **Matrix Mult.** (*shape of W*) $\rightarrow$ **Output shape**

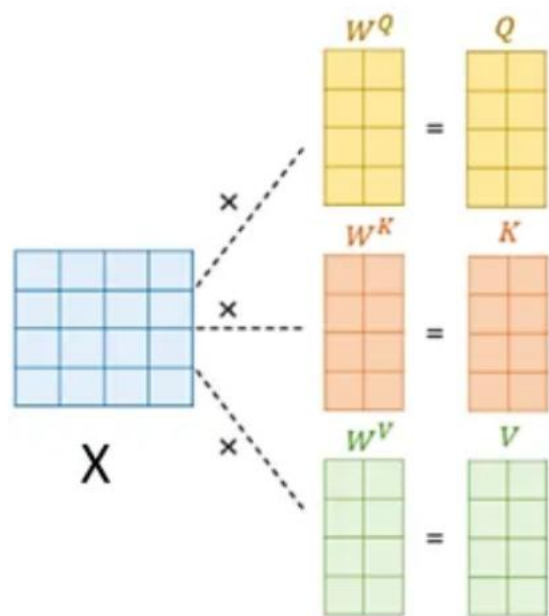
$$Q = XW^Q \rightarrow (T \times d_{model}) * (d_{model} \times d_k) \rightarrow (T \times d_k)$$

$$K = XW^K \rightarrow (T \times d_{model}) * (d_{model} \times d_k) \rightarrow (T \times d_k)$$

$$V = XW^V \rightarrow (T \times d_{model}) * (d_{model} \times d_v) \rightarrow (T \times d_v)$$

Finally, substituting Q, K, V, and $\text{const.} = \sqrt{d_k}$ (d_k dimensionality of the key vector) in above self-attention eqn. (I), we get equation for attention as given below:

$$\text{Attention (Q, K, V)} = \text{SoftMax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \text{ ----- (II)}$$



$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) \times V$$

Figure 5. (Ref.)

Reading Quiz

- On the scale of 1 to 5, rate your understanding of self attention (transformer based), where 1 is the lowest and 5 is the highest.

Shapes as per the paper

<i>Object</i>	<i>Shape</i>	<i>values</i>
q_i, k_i	d_k	(64,)
v_i	d_v	(64,)
x_i	d_{model}	(512,)
W^Q, W^K	$d_{model} \times d_k$	(512, 64)
W^V	$d_{model} \times d_v$	(512, 64)

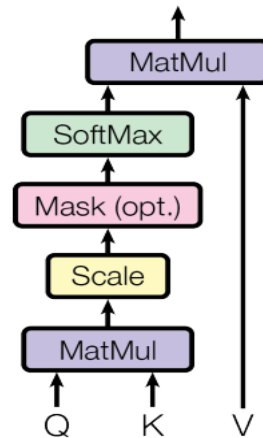
Shape of $QK^T \rightarrow (T \times d_k) \times (d_k \times T) \rightarrow (T \times T)$

And finally, the shape of final attention output: $(T \times T) \times (T \times d_v) \rightarrow (T \times d_v)$

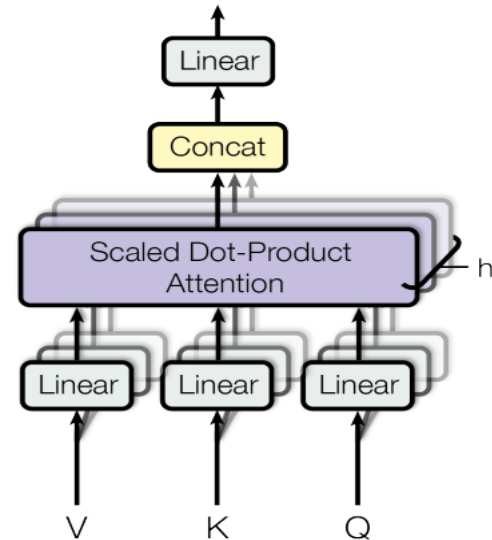
Multi-Head Attention

- “Attention is all you need”: **three trainable weight matrices are introduced and multiplied with input X_i separately, and three new terms Queries(Q), Keys(K), and Values(V) come into the picture.**

Scaled Dot-Product Attention



Multi-Head Attention



Shape of $QK^T \rightarrow (T \times d_k) \times (d_k \times T) \rightarrow (T \times T)$

And finally, the shape of final attention output: $(T \times T) \times (T \times d_v) \rightarrow (T \times d_v)$

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

T X hd_v

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

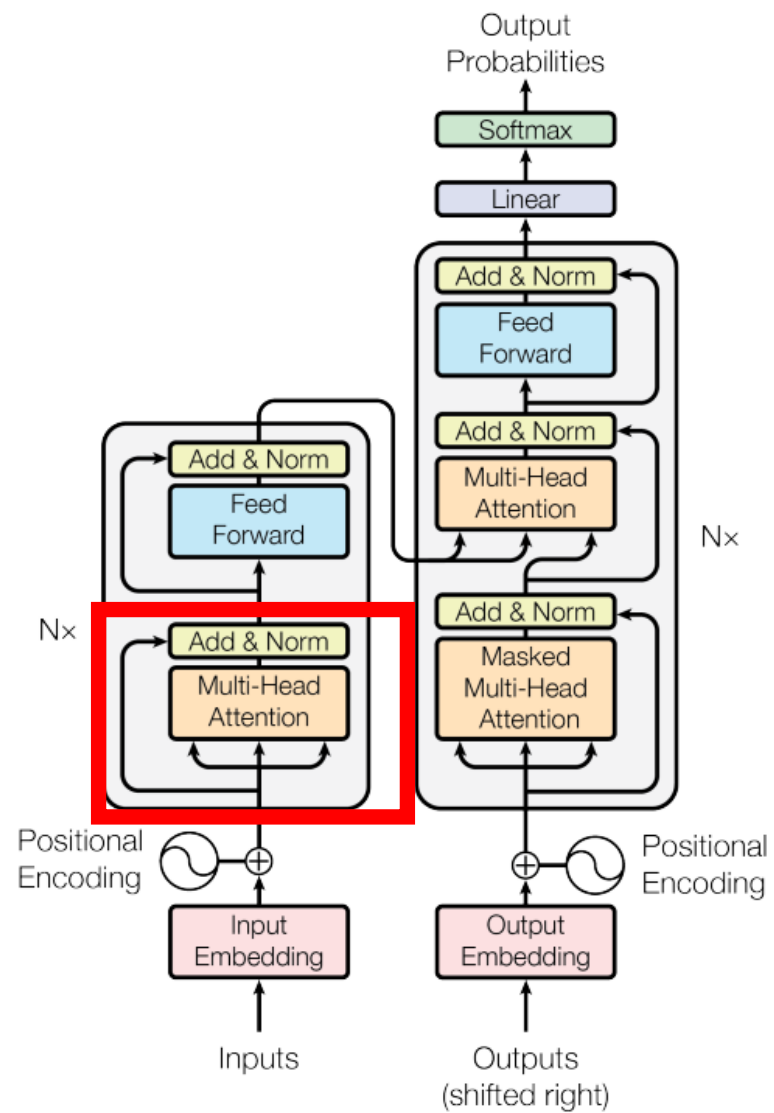


Figure 1: The Transformer - model architecture.

- <https://www.youtube.com/watch?v=QCJQG4DuHT0>
- <https://medium.com/@ramendrakumar/self-attention-d8196b9e9143>
- <https://medium.com/@geetkal67/attention-networks-a-simple-way-to-understand-self-attention-f5fb363c736d>

Transformer (Encoder)

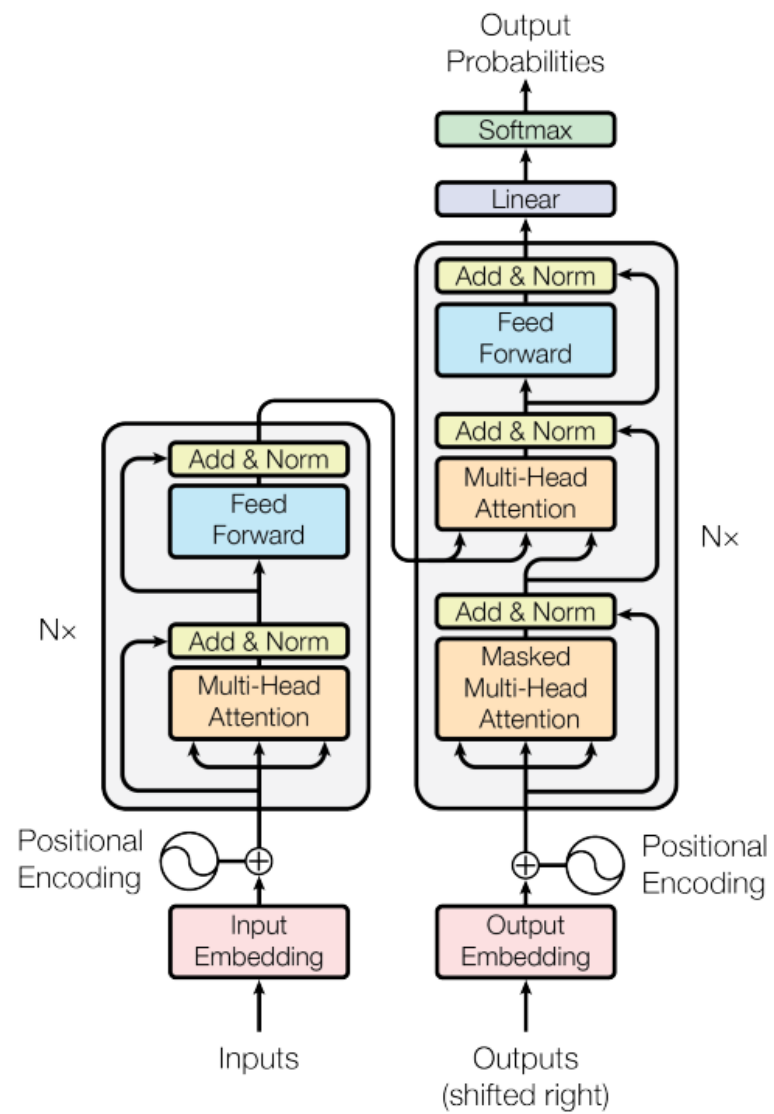


Figure 1: The Transformer - model architecture.

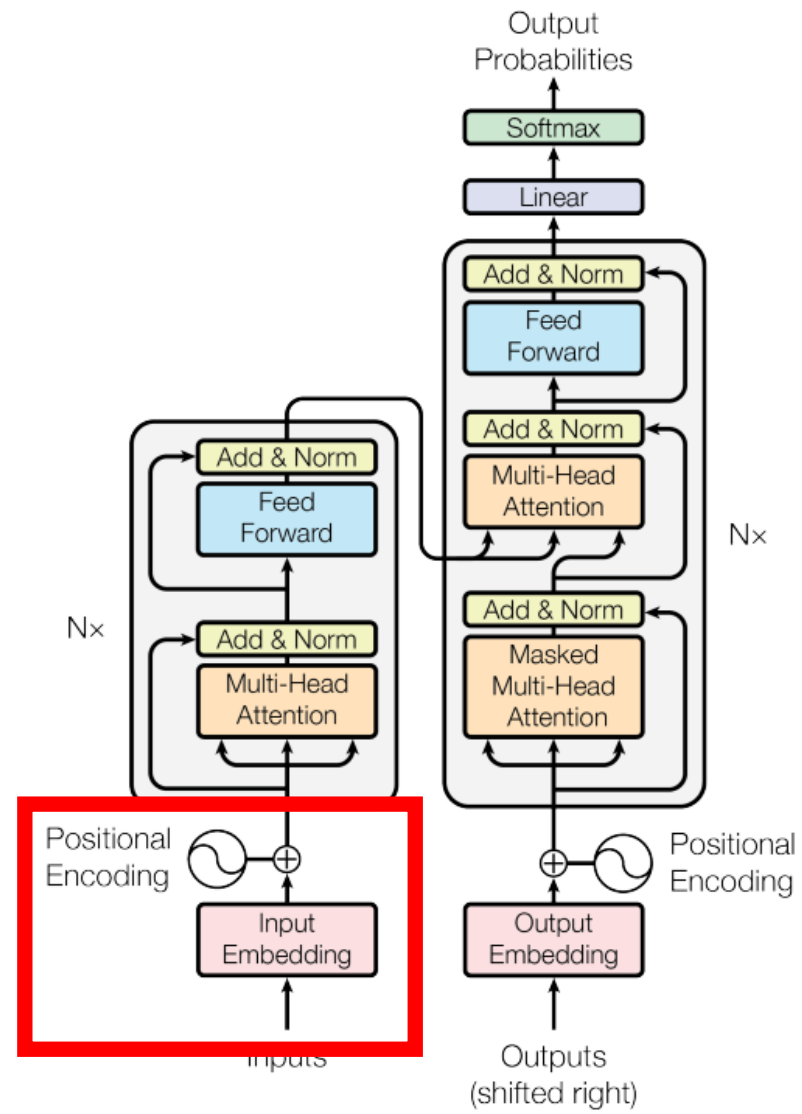


Figure 1: The Transformer - model architecture.

Positional Embedding

“Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed.” [\[From the original paper\]](#)

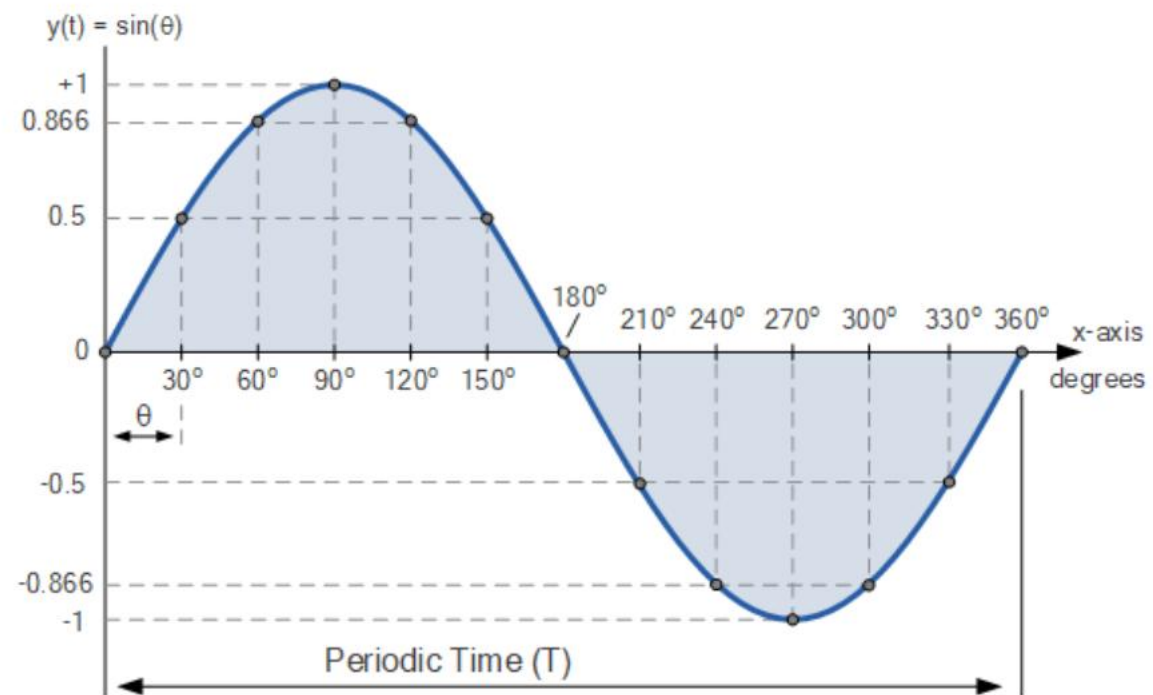
Positional Embedding

In these formulas:

- **pos** is the position (index) of the token in the sequence.
- **d** is the total dimension of the embedding vector.
- **i** represents the specific index within the embedding vector for which the sine or cosine function is being applied.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

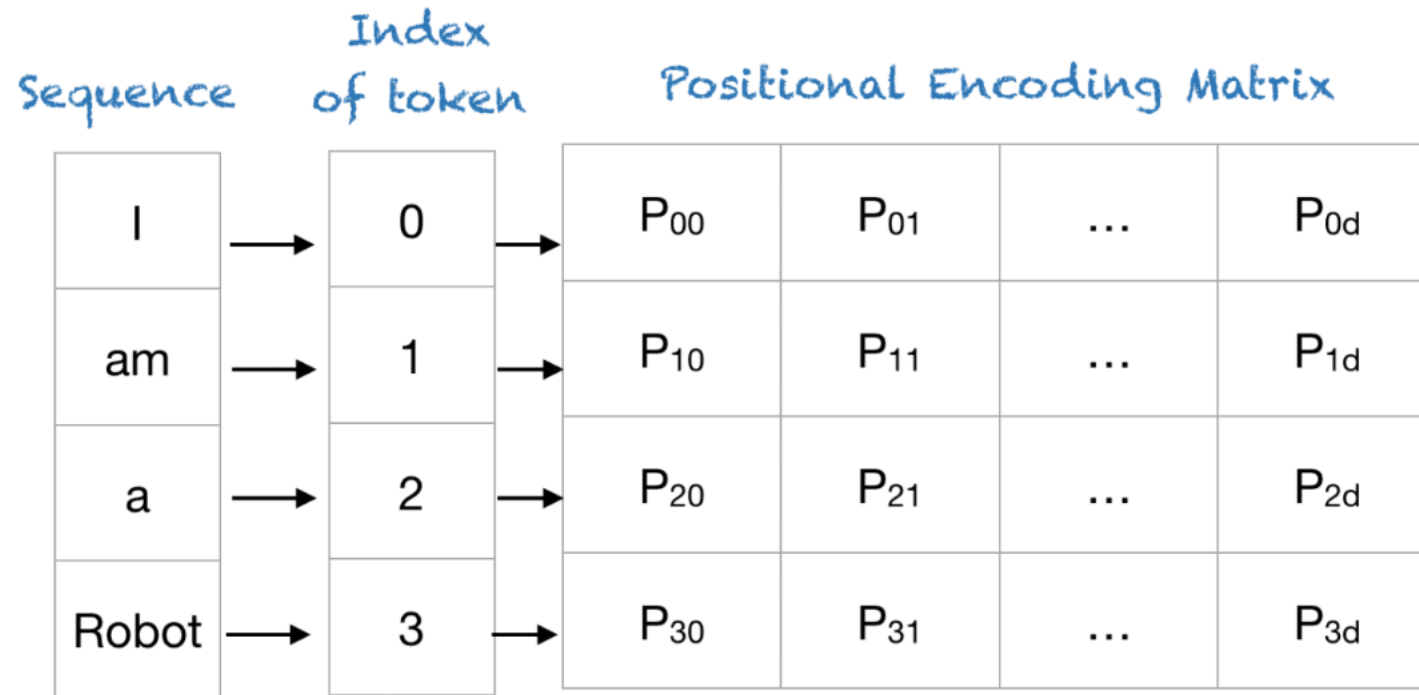


$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

- Please verify values.

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

- For $i = 0$, the denominator $10000^{2i/d} = 1$, so the **angle** is just pos (in radians).
 - $pos = 10$: $\sin(10) \approx -0.544$, $\cos(10) \approx -0.839$
 - $pos = 100$: $\sin(100) \approx -0.506$, $\cos(100) \approx 0.862$
- For $i = 10$, the denominator is ≈ 1.433 . That makes the angle smaller: $pos/1.433$.
 - $pos = 10$: $\sin(6.978) \approx 0.640$, $\cos(6.978) \approx 0.768$
 - $pos = 100$: $\sin(69.783) \approx 0.619$, $\cos(69.783) \approx 0.785$



Positional Encoding Matrix for the sequence 'I am a robot'

<https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>

The positional encoding is calculated for each index i (which ranges from 0 to $d-1$) within the embedding vector for a given position **pos** in the sequence.

- For **even** dimensions (when i is even), the formula uses a sine function:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d}} \right)$$

- For **odd** dimensions (when i is odd), the formula uses a cosine function:

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d}} \right)$$

Sequence	Index of token, k	Positional Encoding Matrix with $d=4$, $n=100$			
		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding

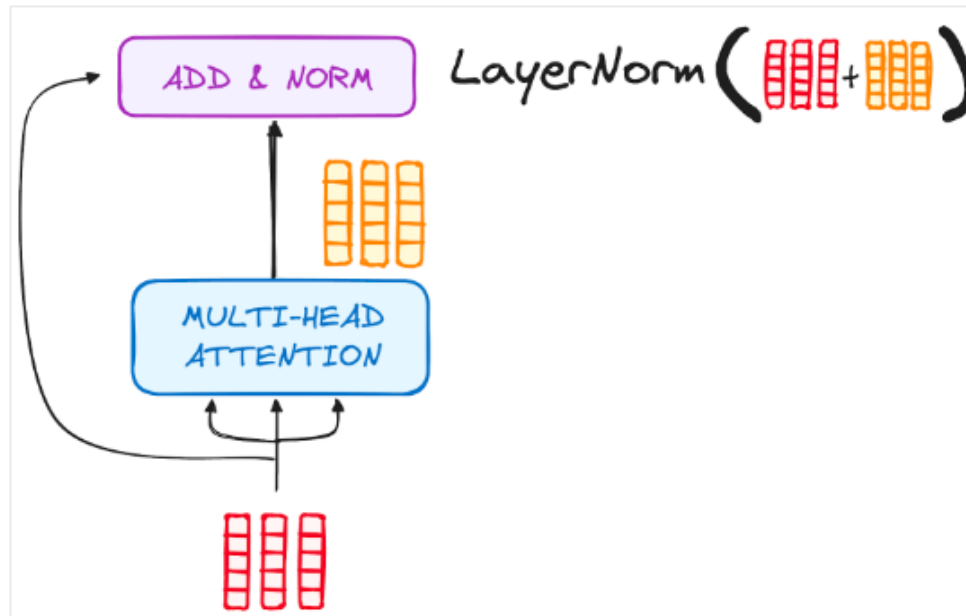
- <https://www.youtube.com/watch?v=T3OT8kqoqjc>

- What should be the dimension of positional embedding?

- <https://www.youtube.com/watch?v=T3OT8kqoqjc>

Add & Norm

Output is added to its input (residual connection) to help mitigate the vanishing gradient problem, allowing deeper models.



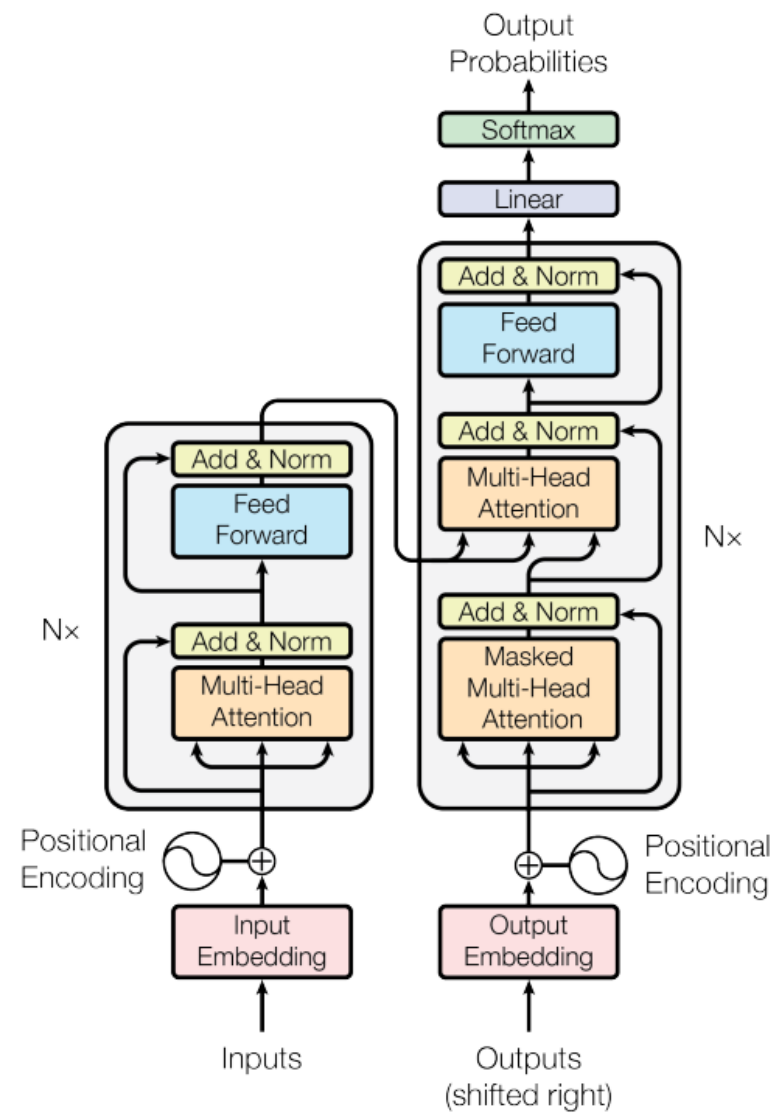
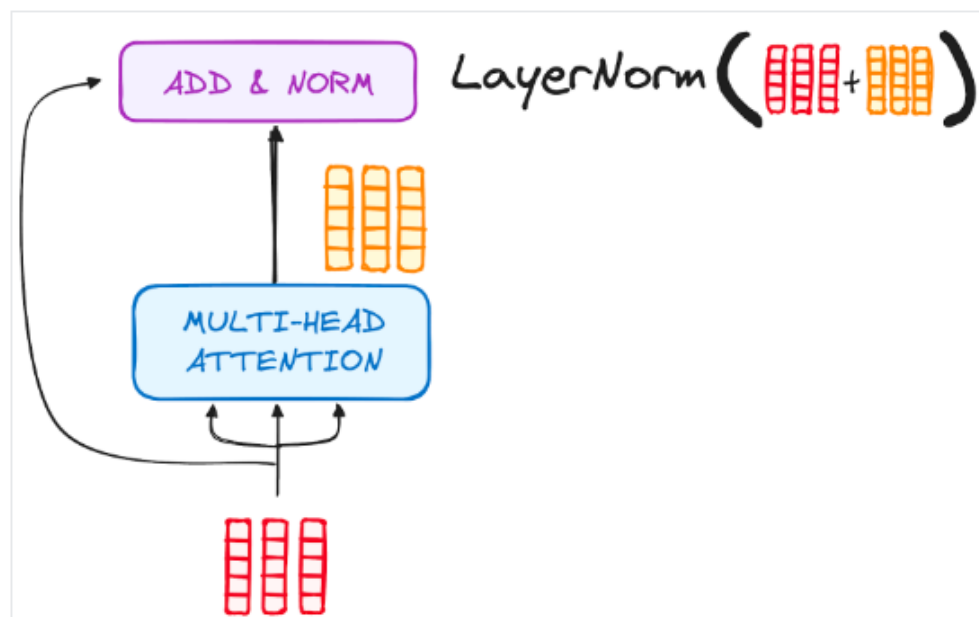


Figure 1: The Transformer - model architecture.

Feed-Forward

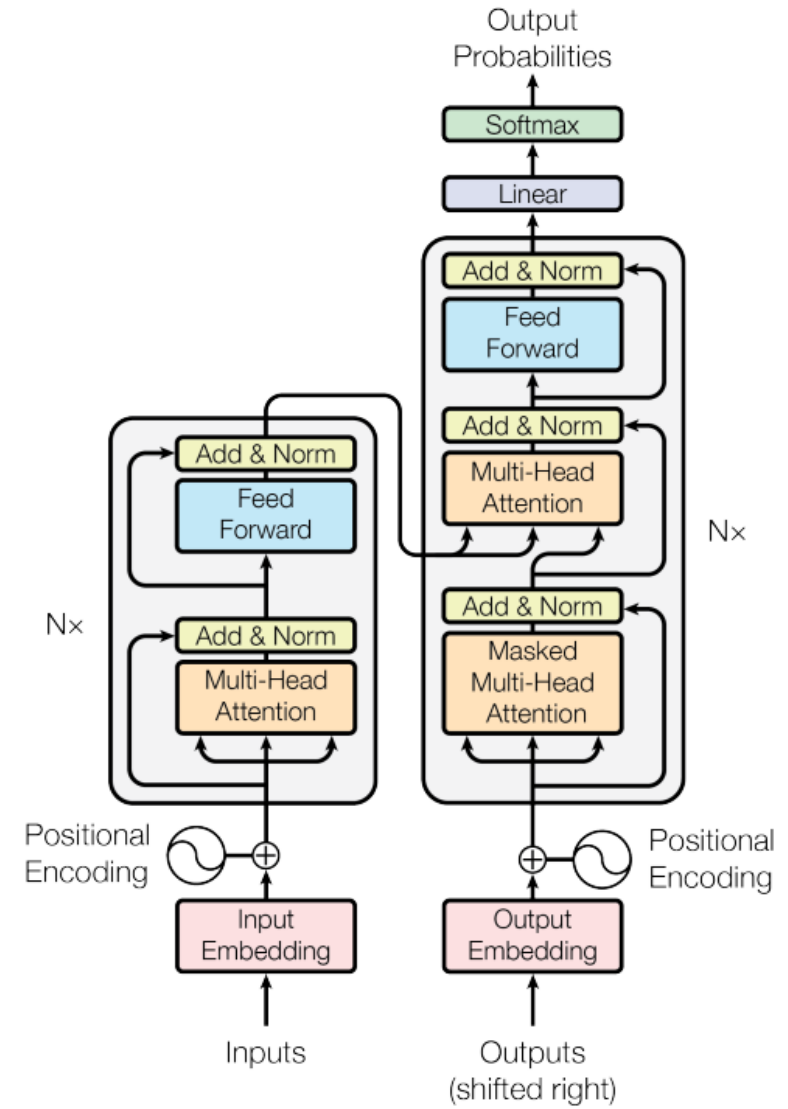
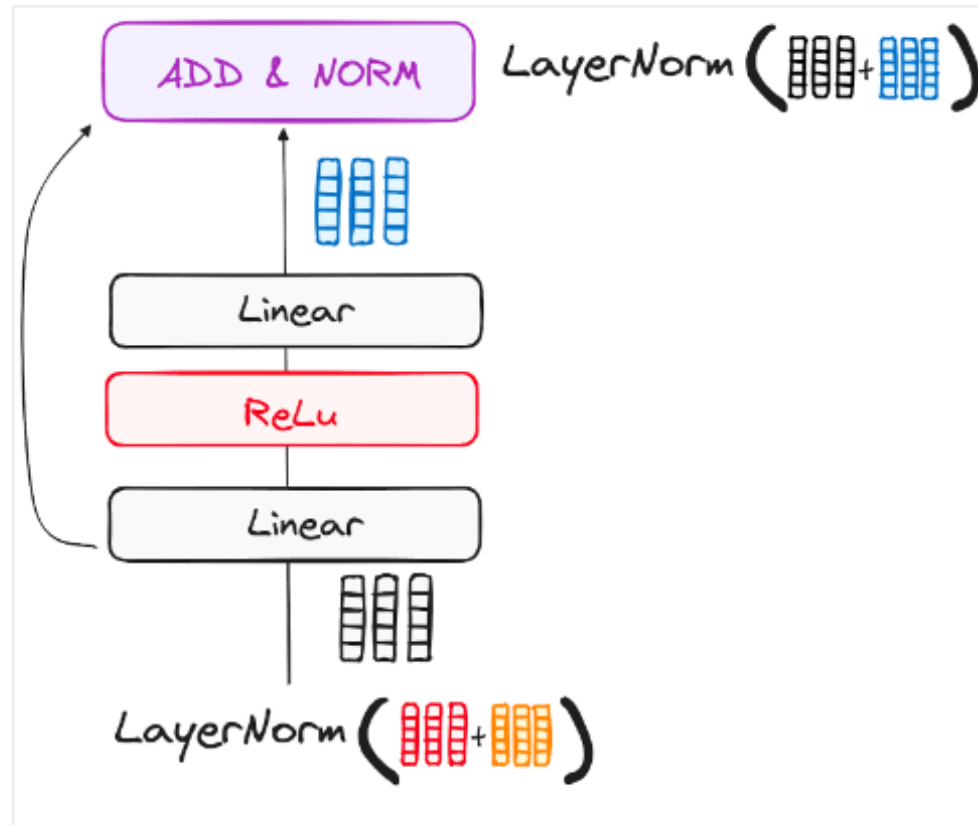
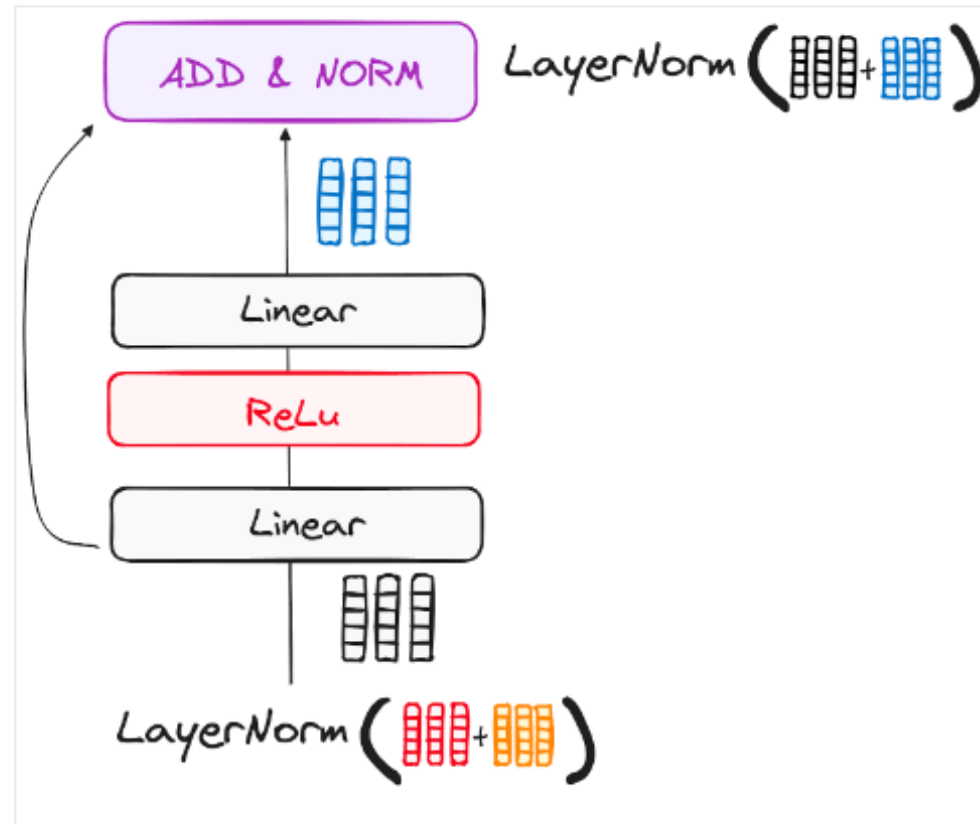


Figure 1: The Transformer - model architecture.

Feed-Forward

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



Transformer (Decoder)

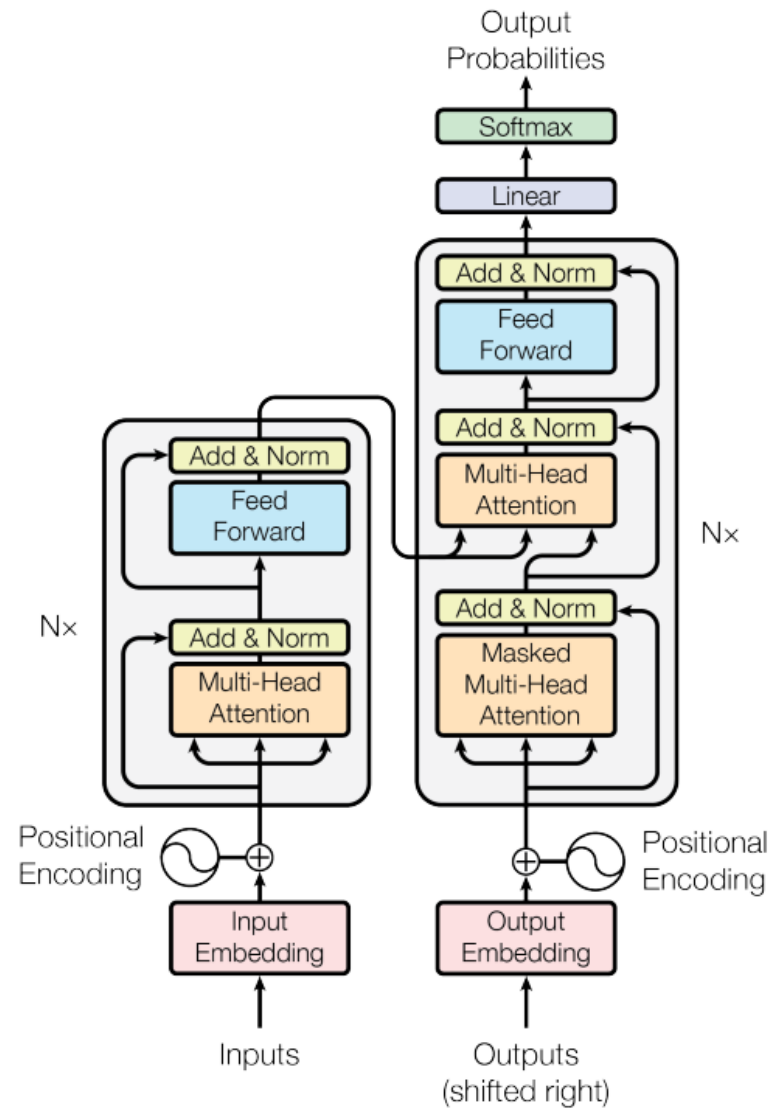
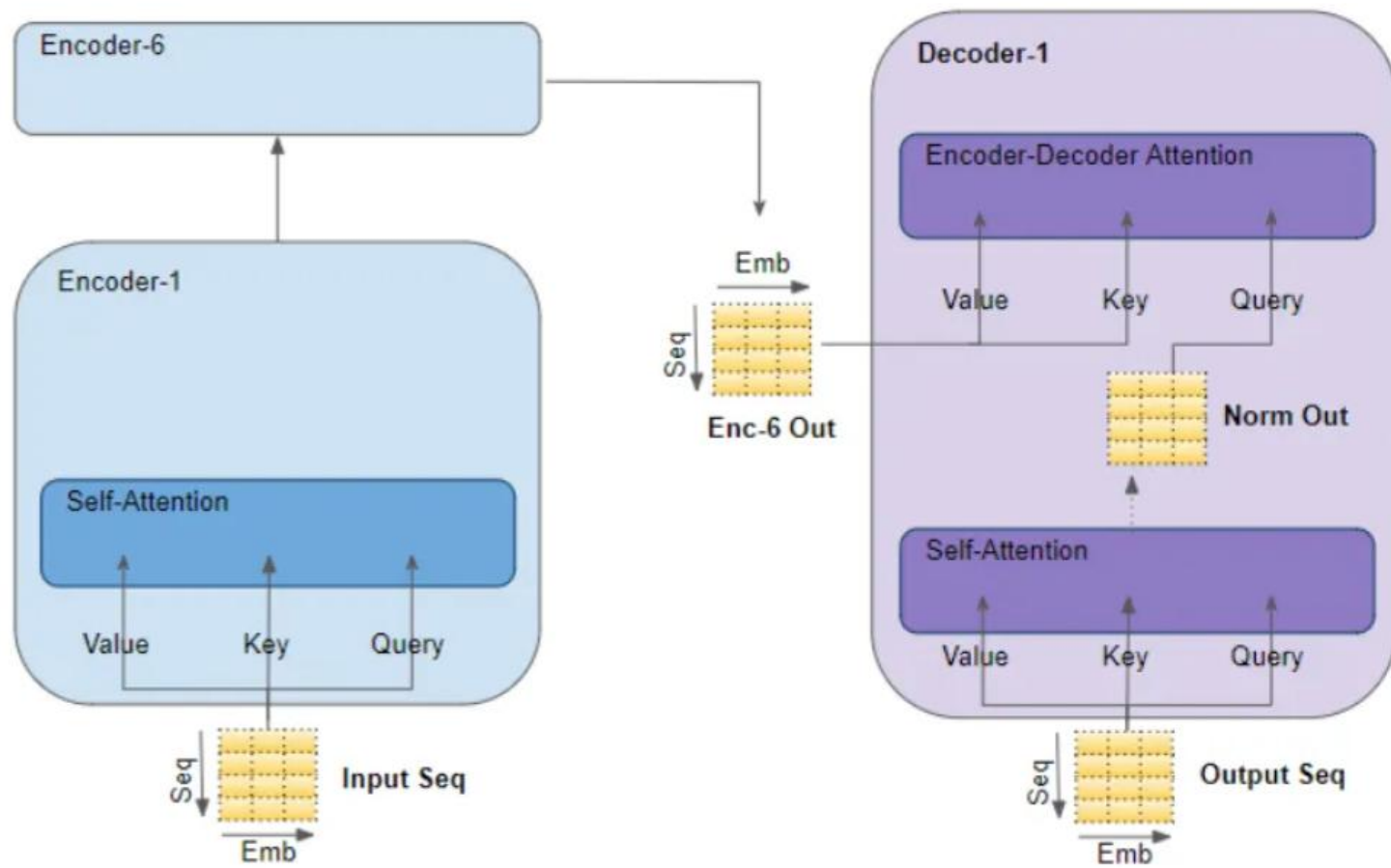


Figure 1: The Transformer - model architecture.



<https://medium.com/@YanAlx/step-by-step-into-transformer-79531eb2bb84>

Attention

Apply Mask here after
finding the dot product

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Masked Multi-head Attention

- The masked multi-head attention in the decoder ensures sequential processing of tokens, a method that prevents each generated token from being influenced by subsequent tokens. This masking is important for maintaining the order and coherence of the generated data.
- We apply masking after finding a dot product between Q and K and before softmax.
 - For first iteration we just add 0 to the output of the dot product.

Example (Masked attention matrix)

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

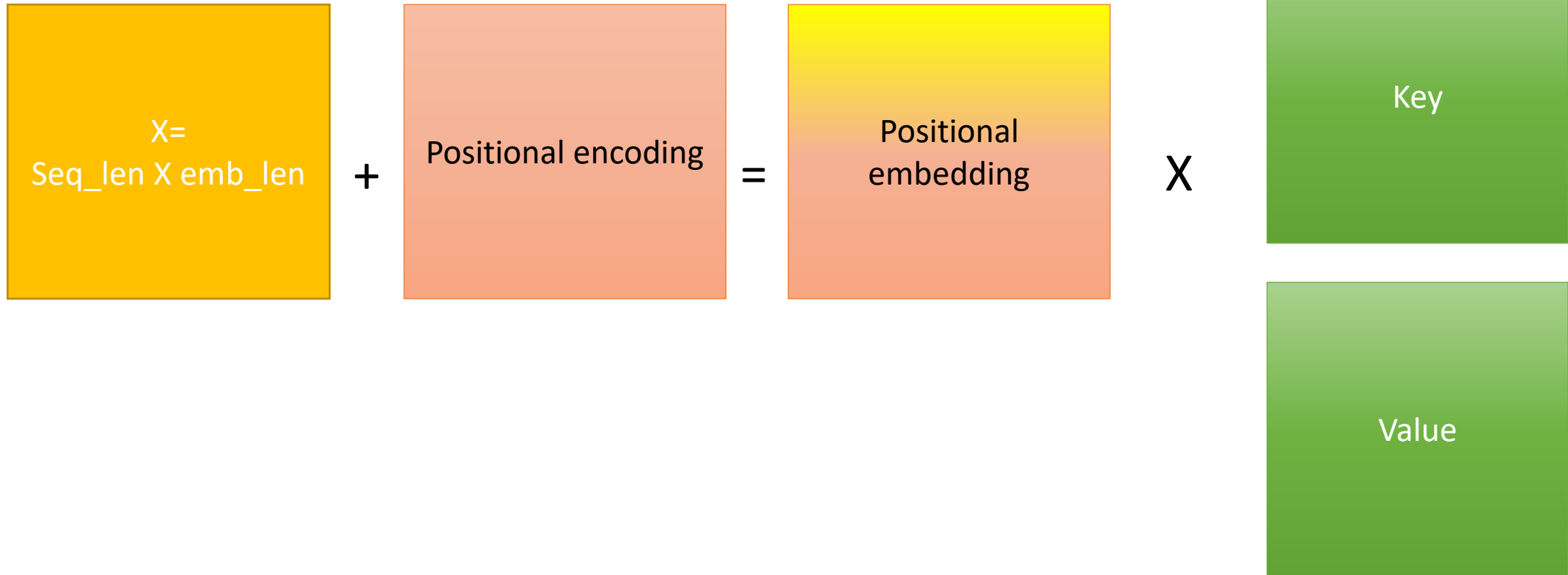
$$\begin{bmatrix} 0.71 & 0.01 & 0.6 \\ 0.77 & 0.34 & 0.34 \\ 0.45 & 0.56 & 0.32 \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.71 & -\infty & -\infty \\ 0.77 & 0.34 & -\infty \\ 0.45 & 0.56 & 0.32 \end{bmatrix}$$

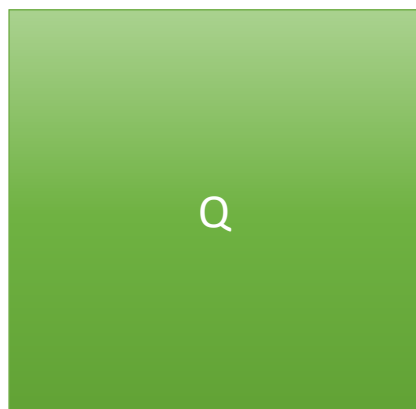
$$\frac{QK^T}{\sqrt{d_k}} + M$$

Dynamic initialization of Mask

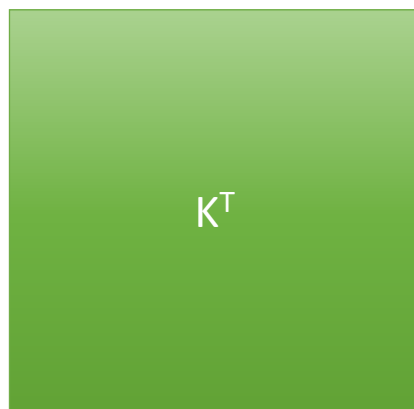
- ```
if Attention_scores.ndim > 1:
 M = np.zeros(Attention_scores.shape)
 for i in range(Attention_scores.shape[0]):
 for j in range(i+1, Attention_scores.shape[1]):
 M[i,j] = -np.inf
else:
 M = 0
```

# Decoder Training

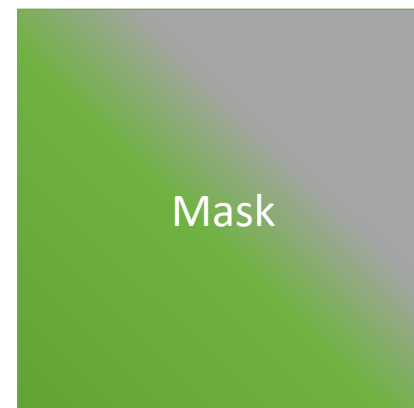




$\times$

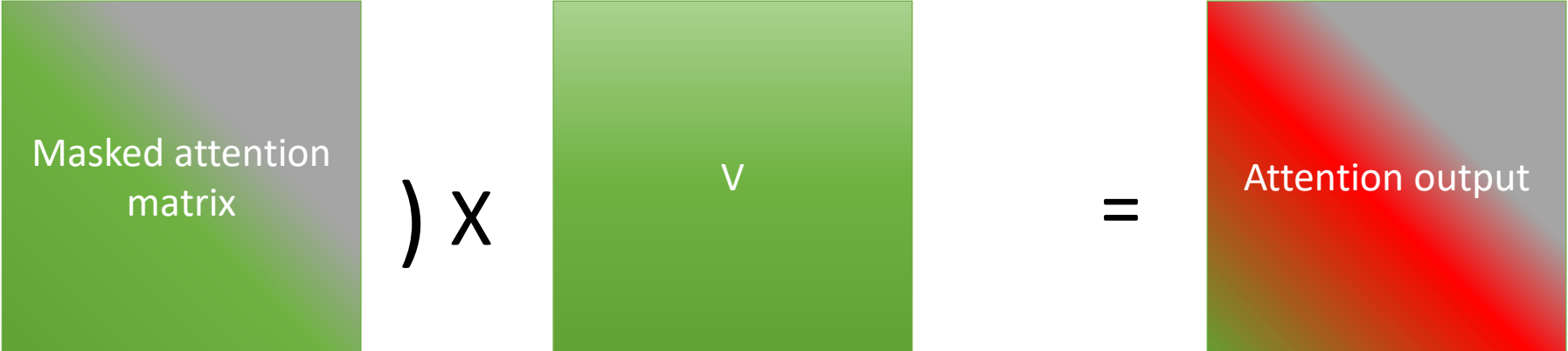


+



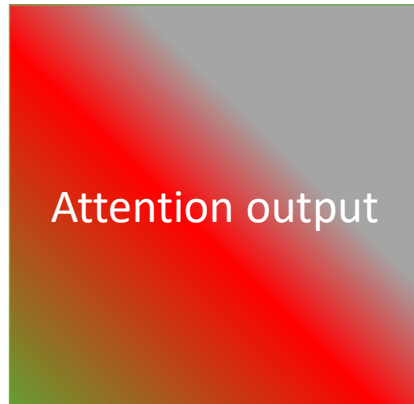
---

$\sqrt{dk}$

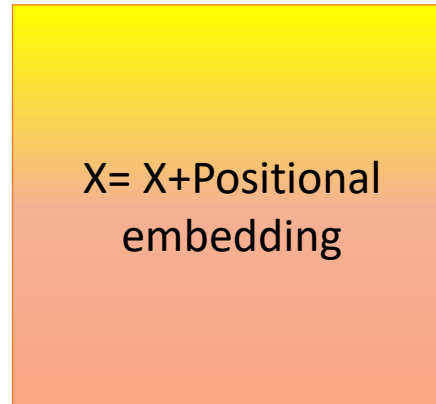
$$\text{Softmax}(\text{Masked attention matrix}) \times V = \text{Attention output}$$


Note: incase of multi-head attention you will replicate a process  $h$  times and concatenate the out and transform the dimension to make it equal to the input dimension.

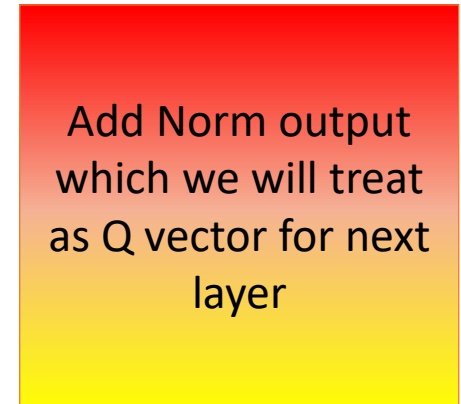
## Add Norm

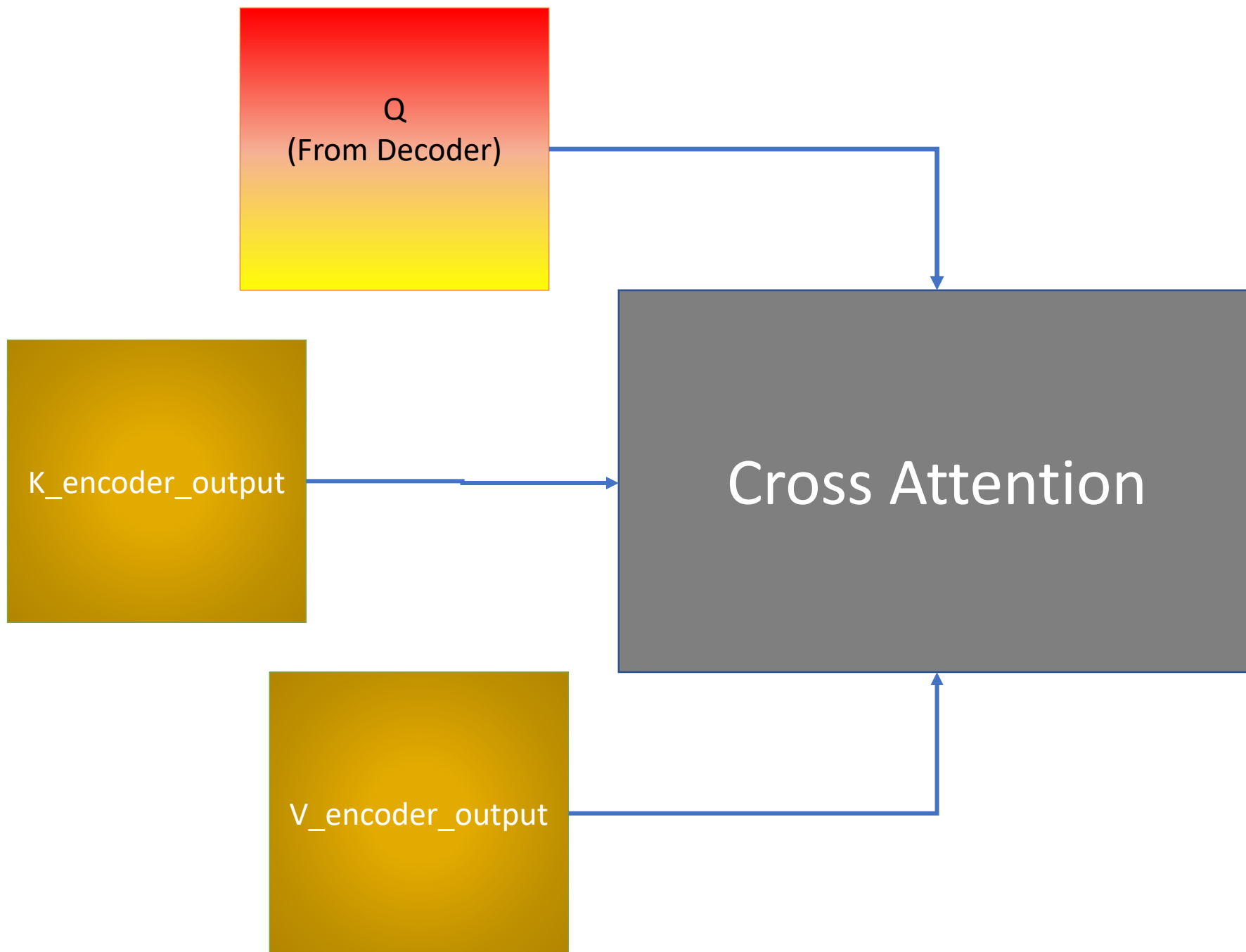


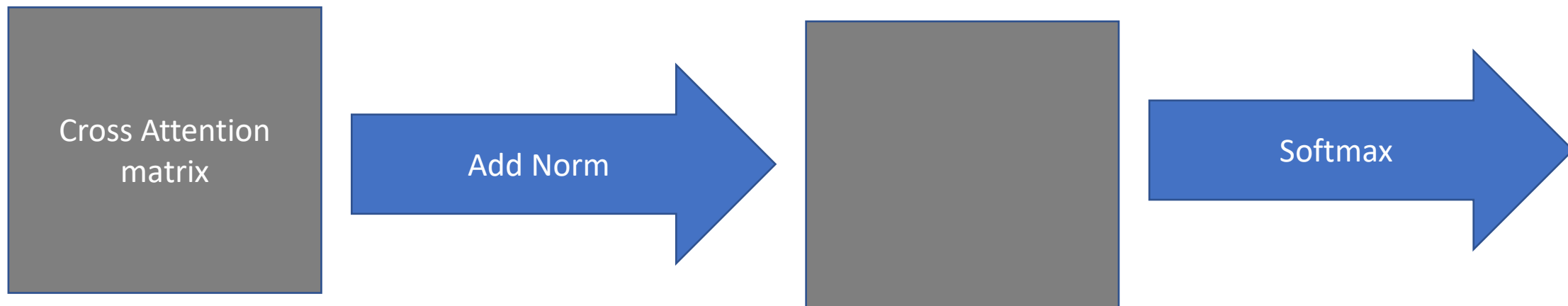
+



=









# Decoder input (Testing)

- First iteration : Input embedding for <Start> token
- Add positional embedding
- Map to three matrices Q, K, V
- Dimensions: 1 X second\_dimension\_QKV

# Decoder output

- A predicted token
- We will append the predicted token after the last input and pass it through the decoder for next prediction.

# Credit: Detailed Explanation

- [Blowing up Transformer Decoder architecture](#)
- <https://zaai.ai/lms-and-transformers-from-scratch-the-decoder/>

# Foundation Models

# Some Famous FMs

- BERT
  - Encoder-only model
  - Classification tasks
- GPT
  - Decoder-only model
  - Sequence-to-sequence tasks or language generation
- LLaMa
  - Decoder-only
  - Sequence-to-sequence tasks or language generation
- T5 & BART
  - Encoder-Decoder model